

Филиал Московского Государственного
Университета имени М.В. Ломоносова в городе
Баку

КУРСОВАЯ РАБОТА

на тему:

«Механизмы преобразования
естественной речи в заданные
шаблоны»

Выполнил:
студент группы 223
Казимов Сеймур Заур оглу

Научный руководитель м.н.с.
Сиротич Богдан Михайлович

Баку - 2026

Содержание

1	Введение	2
1.1	Основные понятия и определения	2
1.2	Описание задачи	2
1.3	История возникновения данной задачи	3
1.4	Компьютерные компоненты на которых реализовывалась программа	4
2	Проектирование системы	4
2.1	Выбор технологий и инструментов	4
2.2	Архитектура программы	5
3	Реализация программы	6
3.1	Общая структура приложения	6
3.2	Модуль распознавания речи	8
3.3	Модуль обработки выражений	9
3.4	Модуль стилизации	10
4	Тестирование и результаты	11
4.1	Как тестировалась программа	11
4.2	Примеры работы программы	11
4.3	Заключение	13

1 Введение

1.1 Основные понятия и определения

Автоматическое распознавание речи (ASR, Automatic Speech Recognition) - технология, позволяющая компьютеру преобразовывать речевой аудиосигнал в текстовое представление.

LaTeX - система компьютерной вёрстки, широко применяемая для оформления научных и технических документов, особенно содержащих математические формулы.

Транскрибация - процесс перевода речевого сигнала в письменный текст.

Скрытая марковская модель (HMM, Hidden Markov Model) - вероятностная модель, используемая для описания временных последовательностей, в которой состояния системы не наблюдаются напрямую.

Промпт (англ. prompt) - это формализованный входной сигнал или текстовая инструкция, подаваемая на вход предварительно обученной языковой модели (LLM) для задания контекста и управления процессом генерации выходной последовательности.

Чанкинг - это процесс разбиения непрерывного входного потока данных (аудиосигнала или длинного текста) на дискретные, логически завершённые сегменты (чанки) фиксированной или переменной длины для обеспечения корректной обработки моделями с ограниченным окном контекста.

1.2 Описание задачи

В этой курсовой разрабатывался способ, который автоматически переводит устную математическую речь в структурированный синтаксис LaTeX. Традиционные системы распознавания голоса обычно хорошо работают с повседневными разговорами и обычным текстом, но с точным научным текстом справляются с трудом. Это связано с тем, что озвучивание формул включает сложные термины, неоднозначность в значении и сложную структуру - например, пределы, кратные интегралы, вложенные индексы и сложные дроби. Поэтому для правильной обработки нужен не простой пословный перевод аудио в текст, а более комплексный подход, который включает семантический анализ и логическое понимание услышанного.

Важность этой разработки связана с особенностями обучения на технических специальностях. Во время лекций студентам приходится одновременно разбираться в сложном материале и точно записывать сложные математические формулы. Часто из-за этого приходится выбирать между пониманием и записыванием конспекта. Автоматизация поможет слушателям сосредоточиться на понимании темы, пока программа возьмёт на себя ведение конспекта.

Основная задача программы - создать инструмент, который принимает аудиозапись с математическими выражениями и выдаёт правильно сформатированный код LaTeX. Итоговый документ должен точно отражать структуру, как если бы его писал студент в тетради. Для этого создаётся сложная система, которая включает:

- Первичную акустическую транскрибацию аудиосигнала в сырой текст.
- Интеллектуальный парсинг транскрибированного текста для выявления математических конструкций и связей между ними.
- Преобразование распознанных логических блоков в команды разметки и финальную сборку готового TeX-шаблона.

1.3 История возникновения данной задачи

Задача преобразования естественной речи в заданные шаблоны (в частности, в структурированный LaTeX-код математических выражений) является логическим развитием технологии автоматического распознавания речи (Automatic Speech Recognition, ASR). История ASR насчитывает более 70 лет и прошла путь от примитивных устройств, способных распознавать отдельные цифры, до современных систем, которые не только транскрибируют речь, но и преобразуют её в формализованные шаблоны с учётом семантики и контекста.

Начало исследований в области ASR относится к 1950-м годам. Первое реальное устройство появилось в 1952 году в лабораториях Bell Laboratories [1]. Система «Audrey» (Automatic Digit Recognizer) была способна распознавать цифры от 0 до 9, произнесённые одним диктором [8]. Она работала на основе анализа энергии в определённых частотных диапазонах (формантах) и требовала предварительной настройки под голос конкретного человека. Хотя «Audrey» была крайне ограниченной, она доказала принципиальную возможность машинного понимания речи.

В 1960-х годах прогресс продолжился. В 1962 году компания IBM представила на Всемирной выставке в Сиэтле устройство «Shoebox», которое распознавало уже 16 английских слов, включая цифры и простые арифметические команды («плюс», «минус», «итого») [2]. «Shoebox» мог выполнять элементарные математические операции по голосовой команде, что стало первым шагом к специализированному распознаванию технической речи. В том же десятилетии появились первые эксперименты с непрерывной речью (без пауз между словами), а советские исследователи разработали алгоритм динамического временного искажения (Dynamic Time Warping, DTW), который позволял работать со словарём до 200 слов.

Настоящий прорыв произошёл в 1970-х годах благодаря государственному финансированию. В 1971-1976 годах Агентство перспективных исследовательских проектов Министерства обороны США (DARPA) запустило программу Speech Understanding Research (SUR) [3]. В рамках проекта были созданы системы Harpy (Carnegie Mellon University) и Hearsay, способные распознавать до 1000 слов в непрерывной речи [3]. Harpy стала одной из первых систем, продемонстрировавших практическую применимость ASR.

В 1980-х годах доминирующей технологией стали скрытые марковские модели (Hidden Markov Models, HMM). Они позволили перейти от простого сравнения шаблонов к вероятностному моделированию речи. Ключевую роль сыграли работы Фреда Джелинека в IBM (система Tangora с словарём 20 000 слов) и команды CMU (Sphinx). HMM заменили DTW и стали основой всех промышленных систем до конца 2000-х годов [14].

Коммерциализация ASR началась в 1990-х. В 1990 году вышла первая потребительская программа Dragon Dictate, а в 1997 году — Dragon NaturallySpeaking, которая поддерживала непрерывную речь со скоростью до 100 слов в минуту. Эти системы уже могли использоваться для диктовки текстов, в том числе технических. В 1992 году AT&T запустила первую голосовую систему маршрутизации звонков [14].

В 2000-х и начале 2010-х годов ASR активно применялась в телефонии, диктовке медицинских заключений и офисных приложениях. Однако точность оставалась недостаточной для сложных доменов, таких как математика. Требовалась не просто транскрипция, а преобразование речи в формализованные шаблоны (LaTeX, MathML и т.д.).

Революция произошла в 2010-х с приходом глубокого обучения. В 2014 году Baidu представила Deep Speech - первую end-to-end модель на основе RNN [4]. В 2015 году Google внедрила Listen, Attend and Spell (LAS) [5], а в 2017 году - трансформеры [6]. Эти технологии резко повысили точность и позволили работать с шумной речью и разными акцентами. В 2022 году OpenAI выпустила модель Whisper [7], которая стала стандартом для многих современных ASR-систем.

Таким образом, с появлением модели Whisper в 2022 году технология автоматического

распознавания речи достигла уровня, позволяющего эффективно решать задачи не только транскрипции, но и преобразования естественной речи в строго заданные шаблоны. Однако, несмотря на первые попытки решения именно математической диктовки ещё в 2009 году (система TalkMaths [9]) и появление новых датасетов и моделей на базе Whisper и LLM в 2024–2025 годах (включая датасет MathBridge [10] и модель MathSpeech [11]), полноценные практические реализации остаются редкостью. Большинство существующих решений либо устарели, либо представляют собой научные прототипы без открытого кода и удобной модульной архитектуры. В связи с этим разработка полноценной системы, преобразующей аудиофайл с математической диктовкой в готовый LaTeX-код с чётко выделенными модулями распознавания, обработки выражений и стилизации, является актуальной и практически значимой задачей.

1.4 Компьютерные компоненты на которых реализовывалась программа

Одним из важных факторов влияющих на реализацию программы является технические характеристики компьютера на котором запускается данная программа. Характеристики компьютера напрямую будут определять какие модели можно будет запускать с приемлемой скоростью и точностью. На более слабом железе (например, без дискретной видеокарты или с малым объёмом видеопамати) придётся использовать более лёгкие модели для транскрибации и для выявления математических формул.

В рамках данной курсовой работы программа разрабатывалась и тестировалась на компьютере с такими техническими характеристиками:

- Процессор: Intel(R) Core(TM) i7-14700HX
- Оперативная память: 16 ГБ
- Видеокарта: NVIDIA GeForce RTX 4060 Laptop GPU (8 ГБ VRAM)

2 Проектирование системы

2.1 Выбор технологий и инструментов

Для реализации системы преобразования естественной математической речи в структурированный LaTeX-код был выбран модульный подход, состоящий из трёх основных этапов: распознавание речи, интеллектуальная обработка текста с преобразованием в математические формулы и финальная стилистическая вёрстка. Такой конвейер позволяет разделить задачи по сложности и использовать наиболее подходящие инструменты для каждой из них.

На первом этапе - автоматического распознавания речи (ASR) - использовалась открытая нейросетевая модель Whisper-medium (OpenAI). Модель была запущена через библиотеку Hugging Face Transformers. Для загрузки и предобработки аудиофайла применялась библиотека librosa. Whisper-medium была выбрана благодаря хорошему качеству распознавания русской технической речи, поддержке чанкинга длинных аудиозаписей и возможности явно указывать язык (`language="russian"`) [12]. Скрипт реализации данного этапа находится в файле `whisper.py`¹.

¹<https://github.com/seymurk95/course-work-speech2math/blob/main/whisper.py>

На втором этапе - преобразования естественного текста в структурированный LaTeX - использовалась локальная большая языковая модель Qwen (конкретно вариант qwen3-esper3-reasoning-coder-instruct-12b-brainstorm20x-i1 в квантизации Q4_K_M). Модель запускалась через сервер LM Studio с использованием OpenAI-совместимого API.

Выбор данной модели обусловлен [13]:

- Высокой компетенцией в работе с формальными языками и LaTeX;
- Хорошим пониманием математической терминологии;
- Возможностью полностью локального запуска.

Обработка выполнялась с помощью тщательно составленного системного промпта, который строго регламентирует правила замены словесных математических конструкций на LaTeX-код, не позволяя модели переписывать исходный текст. Реализация данного модуля содержится в файле qwen.py².

Третий этап - стилистическое оформление LaTeX-документа - также реализован с помощью той же модели Qwen, но с другим системным промптом. Задача модуля заключается в расстановке переносов строк, выделении заголовков жирным шрифтом и логическом разделении абзацев. Данный этап реализован в файле stilist.py³ и позволяет получить аккуратный, готовый к компиляции файл final_lecture.tex.

Общее управление всеми этапами осуществляет главный скрипт main.py⁴, который последовательно запускает модули с помощью модуля subprocess Python. Такой архитектурный выбор упрощает отладку, тестирование и возможную замену отдельных компонентов системы.

В качестве основного языка программирования был выбран Python 3. Для взаимодействия с языковыми моделями использовалась библиотека openai (в режиме совместимости с локальным сервером). Целевым форматом вывода является LaTeX с поддержкой русского языка.

Таким образом, выбранный стек технологий обеспечивает баланс между качеством распознавания и преобразования математической речи, удобством разработки и возможностью запуска на обычном персональном компьютере без зависимости от облачных сервисов.

2.2 Архитектура программы

Архитектура разработанной системы представляет собой линейный конвейер, состоящий из трёх последовательно выполняемых модулей. Такой подход был выбран для обеспечения чёткого разделения ответственности, удобства отладки и возможности независимого тестирования и модификации каждого этапа.

Общая схема работы программы выглядит следующим образом:

- **Входные данные:** аудиофайл (в формате .wav) с записью естественной математической речи.
- **Этап 1. Распознавание речи.** Модуль whisper.py загружает аудиофайл с помощью библиотеки librosa, выполняет предварительную обработку (ресемплинг до 16 кГц) и передаёт данные в модель Whisper-medium. На выходе модуля формируется файл raw_text.txt, содержащий сырой транскрибированный текст.

²<https://github.com/seymurk95/course-work-speech2math/blob/main/qwen.py>

³<https://github.com/seymurk95/course-work-speech2math/blob/main/stilist.py>

⁴<https://github.com/seymurk95/course-work-speech2math/blob/main/main.py>

- **Этап 2. Преобразование в LaTeX.** Модуль `qwen.py` считывает файл `raw_text.txt` и передаёт его в локальную языковую модель Qwen через OpenAI-совместимый API (LM Studio). Модель, руководствуясь жёстким системным промптом, заменяет словесные описания математических выражений на корректные LaTeX-конструкции, нормализует греческие буквы, кванторы, индексы и степени и т.п. математические символы при этом строго сохраняя исходный порядок слов и содержание текста. Результат сохраняется в файл `raw.tex` (полный LaTeX-документ с преамбулой).
- **Этап 3. Стилистическое оформление.** Модуль `stilst.py` принимает файл `raw.tex`, передаёт его в ту же модель Qwen с другим системным промптом. На данном этапе производится финальная вёрстка: расстановка переносов строк, выделение ключевых заголовков, логическое разделение абзацев двойными пустыми строками. Выходной файл — `final_lecture.tex`, готовый к компиляции в PDF.

Управление всем конвейером осуществляется главным скриптом `main.py`. Скрипт последовательно запускает три модуля с помощью модуля `subprocess` Python, проверяет наличие необходимых файлов и выводит статус выполнения каждого этапа. Такой способ организации позволяет легко добавлять новые этапы обработки в будущем.

Архитектура системы обладает следующими преимуществами:

- **Модульность** - каждый компонент может быть заменён или улучшен в зависимости от желания и мощностных качеств компьютера пользователя;
- **Отказоустойчивость** - при ошибке на одном из этапов выполнение останавливается с понятным сообщением;
- **Прозрачность** - промежуточные результаты (`raw_text.txt`, `raw.tex`) сохраняются, что упрощает отладку и анализ ошибок;
- **Локальность** - вся обработка происходит на локальном компьютере без передачи данных в облако.

Схематично архитектура разработанной системы представлена на рисунке 1.

3 Реализация программы

3.1 Общая структура приложения

Приложение реализовано в виде набора самостоятельных Python-скриптов, объединённых главным управляющим модулем `main.py`. Каждый скрипт отвечает за строго определённый этап обработки и может быть запущен как независимо, так и в составе общего конвейера.

Точкой входа в программу является функция `run_pipeline(audio_path)`, определённая в `main.py`. Она принимает путь к аудиофайлу, проверяет его существование и последовательно запускает три модуля обработки: `whisper.py`, `qwen.py`, `stilst.py`. Для запуска каждого модуля используется стандартная библиотека Python `subprocess`, которая вызывает дочерние процессы через тот же интерпретатор Python `sys.executable`. Это гарантирует изоляцию окружения каждого этапа и упрощает обработку ошибок - при сбое любого из модулей выполнение конвейера немедленно прерывается с информативным сообщением.

Важной особенностью реализации является управление ресурсами GPU. Поскольку модель Whisper и языковая модель Qwen не могут одновременно находиться в видеопамяти на имеющемся оборудовании (8 ГБ VRAM), перед запуском транскрибации `main.py`

```

Аудиофайл (в формате .wav)
↓
whisper.py
Whisper-medium (ASR)
↓
raw_text.txt
↓
qwen.py
Qwen (преобразование в LaTeX-формулы)
↓
raw.tex
↓
stilist.py
Qwen (стилистическое оформление)
↓
final_lecture.tex
(Финальный документ)

```

Рис. 1: Архитектура программы преобразования математической речи в LaTeX

выгружает текущую модель из LM Studio. Функция `unload_lm_model()` реализует двух-этапную очистку памяти: сначала предпринимается попытка корректной выгрузки через REST API LM Studio (эндпоинт `/v1/models/unload`), после чего принудительно завершаются все процессы LM Studio с помощью системной команды `kill`. Такой подход гарантирует освобождение видеопамати даже в случае, если API не отвечает или возвращает ошибку. `qwen.py`.

Фрагмент кода модуля `main.py` (управление конвейером и очистка VRAM):

```

def unload_lm_model():
    resp = requests.get(f"{LM_STUDIO_URL}/v1/models")
    if resp.status_code == 200:
        models = resp.json().get("data", [])
        if models:
            model_id = models[0].get("id")
            requests.post(f"{LM_STUDIO_URL}/v1/models/unload",
                json={"model": model_id})
            os.system("kill -9 -f lmstudio")
            time.sleep(2)

def run_pipeline(audio_path):
    unload_lm_model()
    subprocess.run([sys.executable, "whisper.py", audio_path],
        check=True)
    subprocess.run([sys.executable, "qwen.py"], check=True)
    subprocess.run([sys.executable, "stilist.py"], check=True)

```

Взаимодействие между модулями осуществляется через промежуточные текстовые файлы: `raw_text.txt` (результат транскрибации), `raw.tex` (LaTeX с формулами), `final_lecture.tex` (финальный документ). Такой подход позволяет в случае ошибки на каком-либо этапе проверить промежуточный результат и перезапустить обработку с нужного шага, не повторяя все предыдущие.

Файл	Назначение
main.py	Главный управляющий скрипт, оркестратор конвейера
whisper.py	Модуль акустической транскрибации (ASR)
qwen.py	Модуль преобразования текста в LaTeX
stilist.py	Модуль стилистического оформления документа
raw_text.txt	Промежуточный файл: сырой транскрибированный текст
raw.tex	Промежуточный файл: LaTeX с формулами
final_lecture.tex	Финальный выходной документ

3.2 Модуль распознавания речи

Модуль распознавания речи реализован в файле `whisper.py` и отвечает за первый этап конвейера — преобразование входного аудиофайла в сырой транскрибированный текст.

При запуске модуль получает путь к аудиофайлу через аргумент командной строки (`sys.argv[1]`), который передаётся из главного скрипта `main.py`.

Перед загрузкой модели выполняется явная оптимизация памяти GPU: вызывается `torch.cuda.empty_cache()` и `gc.collect()`, а через переменную окружения `PYTORCH_CUDA_ALLOC_CONF` включается режим `expandable_segments`. Это снижает фрагментацию видеопамати, что критично при работе с большими моделями на GPU с ограниченным объёмом VRAM.

Модуль автоматически определяет доступное устройство: если CUDA доступна, используется GPU (`device=0`), в противном случае — CPU (`device=-1`). В зависимости от устройства выбирается тип данных: `float16` для GPU (ускоряет вычисления и снижает потребление памяти) и `float32` для CPU. Модель загружается через интерфейс `pipeline` библиотеки Hugging Face Transformers с указанием реализации механизма внимания `sdpa` (Scaled Dot-Product Attention) — это даёт прирост производительности по сравнению со стандартной реализацией.

Загрузка и предобработка аудио выполняется библиотекой `librosa`. Функция `librosa.load()` вызывается с параметром `sr=16000`, что приводит входной сигнал к частоте дискретизации 16 кГц — именно такой формат ожидает модель Whisper на входе.

Распознавание запускается с рядом параметров, влияющих на качество и производительность:

- `chunk_length_s=30` — аудио разбивается на чанки по 30 секунд, что позволяет обрабатывать записи произвольной длины, не ограниченной контекстным окном модели;
- `stride_length_s=(5, 5)` — перекрытие между соседними чанками в 5 секунд с каждой стороны, которое предотвращает потерю слов на границах сегментов;
- `batch_size=8` — параллельная обработка восьми чанков за один проход, что ускоряет транскрибацию длинных записей;
- `generate_kwargs={"language": "russian", "task": "transcribe"}` — явное указание языка исключает ошибки автоопределения, характерные для технической речи с большим количеством терминов и числовых выражений.

Фрагмент кода модуля `whisper.py`:

```
pipe = pipeline(
    "automatic-speech-recognition",
    model="openai/whisper-medium",
    dtype=torch.float16 if device == 0 else torch.float32,
    device=device,
    model_kwargs={"attn_implementation": "sdpa"}
)

audio_array, sampling_rate = librosa.load(audio_file, sr=16000)

result = pipe(
    audio_array,
    chunk_length_s=30,
    stride_length_s=(5, 5),
    batch_size=8,
    generate_kwargs={"language": "russian", "task": "transcribe"}
)
```

Результатом работы модуля является файл `raw_text.txt` — текст лекции в том виде, в каком его «услышала» модель, без какой-либо постобработки. Этот файл передаётся на вход следующему модулю конвейера — `qwen.py`.

3.3 Модуль обработки выражений

Модуль обработки выражений реализован в файле `qwen.py` и отвечает за второй этап конвейера — преобразование сырого транскрибированного текста из файла `raw_text.txt` в корректно размеченный LaTeX-код.

Взаимодействие с языковой моделью осуществляется через библиотеку `openai` в режиме совместимости с локальным сервером LM Studio. Клиент настраивается с адресом `http://127.0.0.1:1234/v1` и фиктивным ключом `lm-studio`, который требуется библиотекой формально, но не используется для аутентификации. Температура генерации установлена в значение `0.1`, что минимизирует случайность в ответах и обеспечивает стабильный и предсказуемый вывод.

Ключевым элементом модуля является системный промпт, который жёстко регламентирует поведение модели. Промпт запрещает модели переписывать, сокращать или переформулировать исходный текст — она обязана лишь заменять словесные математические конструкции на соответствующие LaTeX-команды, сохраняя порядок слов и содержание. В промпте явно перечислены правила нормализации математики:

- греческие буквы: «гамма» → `$\gamma$` , «альфа» → `$\alpha$` и т.д.;
- кванторы: «для любого» → `$\forall$` , «существует» → `$\exists$` ;
- степени и индексы: «x в квадрате» → `$x^2$` , «a n» → `$a_n$` ;
- запрет на Unicode-символы: $\mathbb{R}$, $\mathbb{Z}$, $\mathbb{Q}$ заменяются на `$\mathbb{R}$` и аналоги.

Промпт также запрещает модели добавлять структурные команды LaTeX (`\documentclass`, `\begin{document}` и т.п.), поскольку финальная сборка документа выполняется самим скриптом. После получения ответа от модели скрипт проверяет наличие тегов `<think>...</think>` — если модель возвращала внутренние рассуждения, они отсекаются и в файл записывается только финальный результат. Дополнительно выполняется

очистка от Markdown-обёрток (“`““latex`”), которые модель может добавить вопреки инструкции.

Фрагмент кода модуля `qwen.py`:

```
completion = client.chat.completions.create(
    model="qwen3-esper3-reasoning-coder-instruct-12b",
    messages=[
        {"role": "system", "content": system_instruction},
        {"role": "user", "content": raw_text}
    ],
    temperature=0.1,
)

content_body = completion.choices[0].message.content
if "<think>" in content_body:
    content_body = content_body.split("</think>")[-1].strip()
    content_body = content_body.replace("““latex", "").replace(
        "““", "").strip()

full_latex_document = (latex_template_start
    + content_body
    + latex_template_end)
```

Финальный документ собирается путём склейки трёх частей: преамбулы с подключением необходимых пакетов (`inputenc`, `fontenc`, `amssymb`, `geometry`), обработанного тела текста и закрывающей команды `\end{document}`. Результат сохраняется в файл `raw.tex`, который передаётся на вход следующему модулю — `stilst.py`.

3.4 Модуль стилизации

Модуль стилизации реализован в файле `stilst.py` и является завершающим этапом конвейера. Он принимает на вход файл `raw.tex`, полученный от модуля `qwen.py`, и выполняет финальное оформление документа, сохраняя результат в файл `final_lecture.tex`, готовый к компиляции в PDF.

Взаимодействие с языковой моделью организовано так же, как и в `qwen.py`: через библиотеку `openai` с подключением к локальному серверу LM Studio. Используется та же модель Qwen с температурой 0.1, однако с принципиально другим системным промптом — модуль выполняет не смысловую обработку, а исключительно вёрстку.

Системный промпт жёстко ограничивает модель пятью запретами: добавлять новые слова, менять формулировки, менять порядок предложений, удалять любые части текста, а также исправлять ошибки в тексте или LaTeX-разметке. Модуль выполняет только три действия:

- расстановку двойных пустых строк между логическими блоками текста (например, между вопросом и ответом);
- выделение жирным шрифтом ключевых структурных сущностей, уже присутствующих в тексте: «Вопрос №», «Ответ», «Определение», «Примеры»;
- сохранение всех математических формул `$...$` и `\[...\]` в неизменном виде.

После получения ответа от модели, аналогично `qwen.py`, выполняется очистка от внутренних рассуждений (теги `<think>`) и Markdown-обёрток. Результат записывается в `final_lecture.tex`.

4 Тестирование и результаты

4.1 Как тестировалась программа

Тестирование программы проводилось методом практической проверки на реальных аудиозаписях математических текстов. Для каждого теста начитывался фрагмент лекции на микрофон, после чего запускал конвейер и оценивался итоговый результат по трём критериям.

Первым критерием являлась успешная компиляция выходного файла `final_lecture.tex` в PDF без критических ошибок. Второй критерий — корректность математических формул: вручную подсчитывалось количество ошибочно преобразованных выражений в итоговом документе. Третий критерий — соответствие результата ожиданиям: полученный PDF сравнивался с тем, как тот же фрагмент был бы оформлен вручную.

В ходе тестирования были записаны аудиофрагменты по нескольким математическим дисциплинам: математическому анализу, дифференциальной геометрии и школьная задача на растворы. Выбор именно этих тем обусловлен их различной сложностью с точки зрения речевого представления формул: если пределы и интегралы содержат относительно стандартные конструкции, то дифференциальная геометрия требует корректного распознавания параметрических кривых и греческих букв. Помимо математических дисциплин университетского уровня, программа была протестирована на простой школьной задаче на проценты. Данный тест преследовал цель проверить, насколько корректно система справляется с элементарными арифметическими выражениями, не перегруженными специальной терминологией. Результат показал, что программа уверенно обрабатывает и такой уровень сложности: числовые выражения и процентные обозначения были корректно оформлены в LaTeX-разметку.

В процессе тестирования была выявлена основная проблема: языковая модель на начальных этапах разработки стремилась перефразировать исходный текст, а не только заменять математические конструкции. Это приводило к тому, что часть исходного содержания искажалась или терялась. Проблема была устранена итеративным уточнением системного промпта в `qwep.ru`: добавлением явных запретов и правил самопроверки перед выводом результата. После нескольких итераций доработки качество преобразования вышло на приемлемый уровень.

4.2 Примеры работы программы

Для иллюстрации работы системы приведём характерный пример из темы математического анализа. Следующий фрагмент демонстрирует входные данные на каждом этапе конвейера и соответствующий выход.

Входная аудиозапись (содержание):фрагмент лекции о множестве нулевой меры: «Вопрос номер 5. Что такое множество нулевой меры на прямой? А на плоскости? привести примеры. Показать, что множество алгебраических чисел бесконечно, но имеет нулевую меру. Ответ. Говорят множество меры нуль $A \in \mathbb{R}$ если его можно накрыть набором интервалов. Возьмём около каждого целого числа интервалы $\frac{\varepsilon}{2^n}$

$$\sum_{k=1}^{\infty} \frac{\varepsilon}{2^k} = \varepsilon$$

. примерами на прямой являются целые числа, рациональные числа, множество из одной точки. Пример 1 прямая, пример 2 рациональные точки. Алгебраические числа выглядят как

$$a_n x^n + \dots + a_1 x + a_0 = 0$$

, каждому полиному соответствует набор целых чисел

$$a_0, \dots, a_n$$

это количество счётно. У полинома n -ой степени не более n корней отсюда следует что для каждого набора n корней, а объединение счётных множеств счетно."

Вывод модуля whisper.py (файл raw_text.txt):

Вопрос №5. Что такое множество нулевой меры на прямой, а на плоскости? Привести примеры. Показать, что множество алгебраических чисел бесконечно, но имеет нулевую меру. Ответ. Говорят, что множество меры 0, а лежащее в $\mathbb{R}$, если его можно накрыть набором интервалов. Вимерами на прямой являются целые числа, рациональные числа, множество из одной точки. Пример один. Прямая. Пр на x , плюс a -ноль, равное нулю. Каждому полиному соответствует набор целых чисел, a -ноль, и т.д., a - n -минус-один, a - n . Это количество счетно. У полинома n -ты ст степени не более n -корней. Отсюда следует, что для каждого набора свои n -корней, а объединение конечного числа счетных множеств счётно.

Вывод модуля qwen.py (тело файла raw.tex):

```
\documentclass[12pt, a4paper]{article}
\usepackage[utf8]{inputenc}
\usepackage[T2A]{fontenc}
\usepackage{amssymb}
\usepackage[left=2cm, right=2cm, top=2cm, bottom=2cm]{geometry}
```

```
\begin{document}
```

Вопрос №5. Что такое множество нулевой меры на прямой, а на плоскости?

Привести примеры. Показать, что множество алгебраических чисел бесконечно, но имеет нулевую меру.

Ответ. Говорят, что множество меры 0 , лежащее в $\mathbb{R}$, если его можно накрыть набором интервалов. Примеры на прямой: целые числа $\mathbb{Z}$, рациональные числа $\mathbb{Q}$, множество из одной точки x_0 .

Пример один. Прямая. Пусть $A = \{a_n\} \subset \mathbb{R}$ - множество корней полинома $P(x) = a_n x^n + \dots + a_1 x + a_0$, где a_i - целые числа. Каждому полиному соответствует набор $\{a_0, a_{-1}, \dots, a_{n-1}\}$, а количество корней не более n . Объединение конечного числа счётных множеств счётно.

Таким образом, множество алгебраических чисел, как объединение счётного числа счётных множеств, имеет нулевую меру.

```
\end{document}
```

Финальный результат после компиляции:

```
\documentclass[12pt, a4paper]{article}
\usepackage[utf8]{inputenc}
```

```

\usepackage[T2A]{fontenc}
\usepackage{amssymb}
\usepackage[left=2cm, right=2cm, top=2cm, bottom=2cm]{geometry}

\begin{document}

\textbf{Вопрос №5.} Что такое множество нулевой меры на прямой,
а на плоскости?
Привести примеры. Показать, что множество алгебраических чисел бесконечно,
но имеет нулевую меру.

Ответ. Говорят, что множество меры  $0$ , лежащее в  $\mathbb{R}$ ,
если его можно накрыть набором интервалов. Примеры на прямой:
целые числа  $\mathbb{Z}$ ,
рациональные числа  $\mathbb{Q}$ ,
множество из одной точки  $x_0$ .

\par\par

Пример один. Прямая. Пусть  $A = \{a_n\} \subset \mathbb{R}$  -
множество корней полинома  $P(x) = a_n x^n + \dots + a_1 x + a_0$ ,
где  $a_i$  - целые числа. Каждому полиному соответствует набор
 $\{a_0, a_{-1}, \dots, a_{n-1}\}$ ,
а количество корней не более  $n$ .
Объединение конечного числа счётных множеств счётно.

Таким образом, множество алгебраических чисел, как объединение
счётного числа счётных множеств, имеет нулевую меру.

\textbf{Примеры}

\end{document}

```

Аналогичным образом тестировалась обработка фрагментов по дифференциальной геометрии, где проверялось корректное распознавание параметрических кривых и греческих букв. Например, произнесённая фраза «кривая гамма от t , где $x(t)$ и $y(t)$ » корректно преобразовывалась в $\gamma(t) = (x(t), y(t))$.

В целом по результатам тестирования система показала приемлемое качество работы для практического применения: математические конструкции стандартной сложности преобразовывались корректно, исходный порядок и содержание текста сохранялись, а итоговый документ успешно компилировался в PDF.

4.3 Заключение

В ходе выполнения данной курсовой работы была разработана система автоматического преобразования естественной математической речи в структурированный LaTeX-код. Поставленная задача была решена в полном объёме: реализован рабочий конвейер, принимающий аудиофайл и выдающий готовый к компиляции .tex-документ.

В процессе работы были решены следующие задачи:

- изучены существующие подходы к автоматическому распознаванию речи и их историческое развитие;
- спроектирована модульная архитектура системы, обеспечивающая чёткое разделение этапов обработки;
- реализованы три независимых модуля: распознавания речи (`whisper.py`), преобразования текста в LaTeX-формулы (`qwen.py`) и стилистического оформления документа (`stilist.py`);
- разработан управляющий скрипт `main.py`, объединяющий модули в единый автоматизированный конвейер с управлением ресурсами GPU;
- проведено тестирование системы на аудиозаписях различной математической сложности — от школьных задач на проценты до тем университетского уровня.

Тестирование показало, что система справляется с преобразованием математических конструкций стандартной сложности: корректно обрабатываются греческие буквы, кванторы, индексы, степени, дроби и символы множеств. Основная проблема, выявленная в ходе разработки — склонность языковой модели к перефразированию исходного текста — была успешно устранена итеративным уточнением системного промпта.

Практическая значимость разработки состоит в том, что система полностью локальна, не требует подключения к облачным сервисам и может быть запущена на обычном персональном компьютере. Модульная архитектура позволяет легко заменять отдельные компоненты — например, использовать более лёгкие модели на слабом оборудовании или более мощные при наличии соответствующих ресурсов.

В качестве направлений дальнейшего развития системы можно выделить: добавление графического интерфейса для удобства использования, поддержку потоковой обработки аудио в реальном времени, а также расширение промпта для работы с более сложными математическими конструкциями, такими как коммутативные диаграммы и построения графиков.

Список литературы

- [1] Davis K.H., Biddulph R., Balashek S. Automatic recognition of spoken digits // Journal of the Acoustical Society of America. — 1952. — Vol. 24. — P. 637–642.
- [2] IBM Shoebox // HandWiki. — URL: https://handwiki.org/wiki/Software:IBM_Shoebox.
- [3] Reddy D.R. et al. A Historical Perspective of Speech Recognition // Communications of the ACM. — URL: <https://cacm.acm.org/research/a-historical-perspective-of-speech-recognition/>.
- [4] Hannun A. et al. Deep Speech: Scaling up end-to-end speech recognition // arXiv preprint. — 2014. — arXiv:1412.5567. — URL: <https://arxiv.org/abs/1412.5567>.
- [5] Chan W. et al. Listen, Attend and Spell // arXiv preprint. — 2015. — arXiv:1508.01211. — URL: <https://arxiv.org/abs/1508.01211>.
- [6] Vaswani A. et al. Attention Is All You Need // Advances in Neural Information Processing Systems (NeurIPS). — 2017. — Vol. 30.

- [7] Radford A. et al. Robust Speech Recognition via Large-Scale Weak Supervision // OpenAI Technical Report. — 2022. — URL: <https://openai.com/research/whisper>.
- [8] Audrey, Alexa, HAL and More // Computer History Museum. — URL: <https://computerhistory.org/blog/audrey-alexa-hal-and-more/>.
- [9] Wigmore A., Hunter G., Pflügel E., Denholm-Price J., Binelli V. Using Automatic Speech Recognition to Dictate Mathematical Expressions: The Development of the 'TalkMaths' Application at Kingston University // Journal of Computers in Mathematics and Science Teaching. — 2009. — Vol. 28, No. 2. — P. 177–189.
- [10] Jung K., Hyeon S., Kwon J.Y., Kim N.-J., Ryu H.G., Lee H.-J., Do J. MathBridge: A Large Corpus Dataset for Translating Spoken Mathematical Expressions into LaTeX Formulas for Improved Readability // arXiv preprint. — 2024. — arXiv:2408.07081. — URL: <https://arxiv.org/abs/2408.07081>.
- [11] Hyeon S., Jung K., Won J., Kim N.-J., Ryu H.G., Lee H.-J., Do J. MathSpeech: Leveraging Small LMs for Accurate Conversion in Mathematical Speech-to-Formula // Proceedings of the AAAI Conference on Artificial Intelligence. — 2025. — Vol. 39. — P. 24194–24202.
- [12] Hugging Face openai/whisper-medium <https://huggingface.co/openai/whisper-medium>.
- [13] Hugging Face mradermacher/Qwen3-Esper3-Reasoning-CODER-Instruct-12B-Brainstorm20x-i1-GGUF
<https://huggingface.co/mradermacher/Qwen3-Esper3-Reasoning-CODER-Instruct-12B-Brainstorm20x-i1-GGUF>.
- [14] Huang X., Baker J., Reddy R. A Historical Perspective of Speech Recognition // Communications of the ACM. 2014. Vol. 57, No. 1. P. 94–103. DOI: 10.1145/2500887. <https://dl.acm.org/doi/epdf/10.1145/2500887>



Исходный код проекта на GitHub